CME 213 – Milestone 3

CUDAsearch: Distributed GPU-Accelerated Dense Vector Similarity Search

Ava Kouhana · Ethan Cohen

Code repository: github.com/Ethancohenn/cudasearch

1 Summary of progress

Since Milestone 2, we have completed the single-GPU version of CUDAsearch. The system performs exhaustive maximum inner-product search (MIPS): for a query batch $Q \in R^{B\times d}$ and database $X \in R^{N\times d}$, it computes $S = QX^\top$ and returns the top-k database rows for each query.

The repository now contains a CPU/OpenMP baseline, dataset loaders for SIFT1M and GIST1M, a benchmark driver, correctness tests, and four CUDA search paths: naive FP32, tiled FP32, row-major INT8, and a cached tiled INT8 implementation.

There are two changes relative to our Milestone 2 design. First, top-k selection is still done on the CPU after copying the full score matrix back from the GPU. This kept the implementation simple and made all backends use the same selection code. Second, after seeing that the first INT8 kernel was slower than expected, we added a better INT8 version, `int8_tiled`, that caches the quantized database on the GPU and uses a transposed INT8 layout for coalesced loads.

2 CUDA kernels

All CUDA code is in `src/cuda/gpu_search.cu`.

Naive FP32. `gpu_search_naive_kernel` launches one CUDA thread per query-database pair. Thread (b, n) loops over all d dimensions, computes $\langle q_b, x_n\rangle$, and writes one score. Global-memory reuse is poor: the same query and database values are reread many times.

Tiled FP32. `gpu_search_tiled_kernel` uses 32×32 thread blocks. Each block computes a 32×32 tile of the score matrix. In each step along the embedding dimension, the block loads a tile of Q and a tile of X into shared memory, synchronizes, and reuses those values for 32 multiply-adds per thread. This addresses the main weakness of the naive kernel: repeated global-memory reads.

INT8 variants. `gpu_search_int8_kernel` stores each database coordinate as int8 with one scale per row, while keeping queries and scores in FP32. This reduces database storage by $4\times$, but the first version quantizes inside each search call and uses a row-major layout, causing strided, poorly coalesced loads. The improved `GpuInt8TiledIndex` quantizes once, keeps the database cached on the GPU, stores it transposed as `Xq_T[dim][row]` for coalesced loads, and reuses scratch buffers across trials.

3 C++ wrapper and CPU code

The benchmark driver `bench/main_bench.cpp` loads the dataset, selects the backend, runs timed trials, and reports latency, throughput (QPS), and Recall@k. The CUDA wrappers allocate device memory, copy inputs, launch the kernel, copy the $B \times N$ score matrix back to the host, and run CPU top-k selection using `std::nth_element`. The cached INT8 wrapper differs by quantizing and copying the database to the GPU once at index construction time. The CPU baseline in `src/core/cpu_search.cpp` is an exhaustive OpenMP scan over database vectors and uses the same top-k selection code as the GPU paths.

4 Correctness testing

Correctness is tested in `tests/test_recall.cpp`. On synthetic normalized data, we compute exact ground truth and obtain Recall@10 = 1.0000 for the CPU, naive CUDA, and tiled CUDA paths, while INT8 reaches 0.9890 due to quantization. On SIFT1M, FP32 paths return Recall@10 around 0.990 and INT8 returns 0.9630. On GIST1M, all implementations agree closely: FP32 gives 0.3560 and INT8 gives 0.3540. The lower GIST1M value is not a CPU/GPU mismatch, but comes from comparing our normalized inner-product objective with the dataset's original benchmark ground truth.

5 Preliminary performance

Benchmarks were run on one Quadro RTX 6000 GPU on a Stanford HPCC `gpu-turing` node, with 16 allocated CPU cores. All rows use $N = 10^6$, $B = 100$, $k = 10$, and five timed trials. For the cached INT8 index, the one-time database quantization and index construction are excluded from the timed search loop, matching how a vector-search index would be used in practice.

Dataset	Backend	Mean ms	QPS	Recall@10	Speedup vs CPU
SIFT1M	CPU-16	1107.9	90.3	0.9900	1.00×
SIFT1M	CUDA naive	1046.8	95.5	0.9900	1.06×
SIFT1M	CUDA tiled	772.2	129.5	0.9900	1.43×
SIFT1M	CUDA INT8 row-major	1893.2	52.8	0.9630	0.59×
SIFT1M	CUDA INT8 tiled	**699.3**	**143.0**	0.9630	**1.58**×
GIST1M	CPU-16	7506.8	13.3	0.3560	1.00×
GIST1M	CUDA naive	8926.4	11.2	0.3560	0.84×
GIST1M	CUDA tiled	1541.6	64.9	0.3560	4.87×
GIST1M	CUDA INT8 row-major	6497.2	15.4	0.3540	1.16×
GIST1M	CUDA INT8 tiled	**1031.7**	**96.9**	0.3540	**7.28**×

Table 1: Single-GPU performance on SIFT1M and GIST1M.

Throughput, GFLOPS, and efficiency. Each batch computes BN dot products. Since each dot product has length d, the work is approximately $2BNd$ FLOPs. For SIFT1M ($d = 128$), this is 2.56×10^{10} FLOPs per batch; for GIST1M ($d = 960$), it is 1.92×10^{11} FLOPs. Thus, GIST1M has lower Queries Per Sec, but much more work per query.

Dataset	Backend	M dot/s	Effective GFLOP/s	FP32-equivalent peak efficiency
SIFT1M	CUDA naive	95.5	24.5	0.15%
SIFT1M	CUDA tiled	129.5	33.2	0.20%
SIFT1M	CUDA INT8 tiled	143.0	36.6	0.22%
GIST1M	CUDA naive	11.2	21.5	0.13%
GIST1M	CUDA tiled	64.9	124.5	0.76%
GIST1M	CUDA INT8 tiled	96.9	186.1	1.14%

Table 2: Derived end-to-end throughput. "M dot/s" is million query-database dot products per sec. These are end-to-end numbers including transfers, score copy-back, and host top-k, so they are lower than pure kernel throughput.

Mdot/s is computed as $(B/t)N/10^6$, where t is the mean runtime in seconds; effective GFLOP/s is then Mdot/s $\cdot\, 2d/1000$ because each length-d dot product uses about $2d$ operations; and peak efficiency is compared to the RTX 6000 FP32 peak of about 16.3 TFLOP/s.

The very low peak efficiencies do not mean that the GPU arithmetic units are the bottleneck. Instead, they show that the end-to-end pipeline is dominated by memory movement and CPU-side selection. Each search materializes a $B \times N$ score matrix. This is 10^8 FP32 scores, or about 400 MB copied back to the host before top-k selection. This fixed copy-back cost is especially important on SIFT1M, where the dot products are short.

Memory-bound versus compute-bound. The roofline model predicts that these kernels are memory-bound. On the RTX 6000, the ridge point is approximately $\frac{16.3 \text{ TFLOP/s}}{672 \text{ GB/s}} \approx 24.3$ FLOP/byte.

The naive FP32 kernel is far below this threshold: it repeatedly streams query and database vectors from global memory and performs only two FLOPs per coordinate. This explains why naive CUDA barely improves over the 16-thread CPU baseline on SIFT1M and is slower on GIST1M.

The tiled FP32 kernel improves arithmetic intensity by loading 32×32 tiles into shared memory and reusing them. Ideally, one tile performs $2 \cdot 32^3$ FLOPs while loading roughly two 32×32 FP32 tiles, giving about 8 FLOP/byte. This is much better than the naive kernel, but still below the ridge point, so the tiled kernel remains memory-bound. The measured speedups are consistent with this: tiling gives only 1.36× over naive CUDA on SIFT1M, but 5.79× on GIST1M, where the larger dimension $d = 960$ makes data reuse more valuable.

INT8 and memory layout. The INT8 results show that precision reduction helps only if memory accesses are efficient. The row-major INT8 kernel is slow because it quantizes inside each search call and because neighboring threads read strided INT8 locations, leading to poor coalescing. The cached tiled INT8 kernel fixes this by quantizing once, keeping the database on the GPU, and storing it transposed as `Xq_T[dim][row]`. It is therefore the fastest backend on both datasets: 2.71× faster than row-major INT8 on SIFT1M and 6.30× faster on GIST1M. It also beats FP32 tiled by 1.10× on SIFT1M and 1.49× on GIST1M.

Overall, the kernels are not mainly latency-bound: the batch contains $BN = 10^8$ independent output scores, which provides enough parallelism to hide much of the memory latency. The main limitation is memory bandwidth and memory traffic. Thus, the most important next optimization is to avoid copying the full score matrix back to the CPU by doing top-k selection on the GPU.

6 Challenges and next steps

The main challenge was that the first INT8 kernel was slower than expected due to per-call quantization and strided row-major memory accesses. We fixed this with a cached, transposed INT8 index. For M4, we will implement MPI sharding across multiple GPUs: split the database by rows, compute local top-k per GPU, and merge local results into a global top-k. For the final report, we also plan to move top-k selection to the GPU, use Nsight Compute, and compare against cuBLAS or FAISS if time permits.